Embedded Programming

Deep Learning Processing Unit (DPU) and Vitis AI

Dr. Angela Cratere
Department of Advanced Computing Sciences
Maastricht University
2025-2026
angela.cratere@maastrichtuniversity.nl

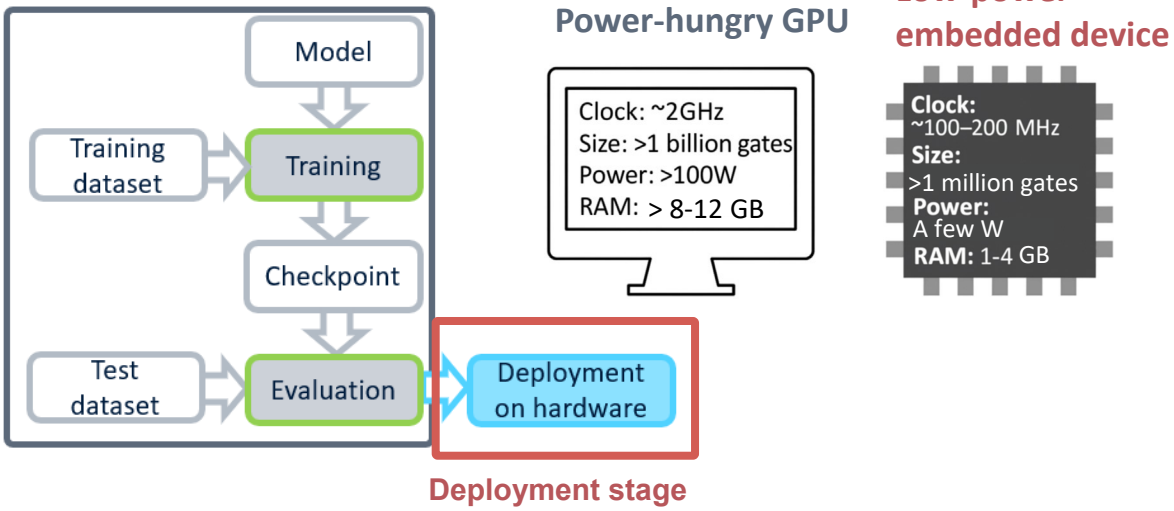
Takeaways from the last lecture

- What is an FPGA and what enables its programmability?
- What is a LUT and how can it be used to implement a combinational function?
- How to program an FPGA
- Neural Networks
- From training to deployment on edge devices: the edge AI challenges

Outline

- Edge AI deployment challenges
- The Xilinx DPU architecture
- The Vitis AI toolchain
- Quantization fundamentals
- Quantization and compilation in Vitis AI

The Edge AI Challenges



Deployment Challenges

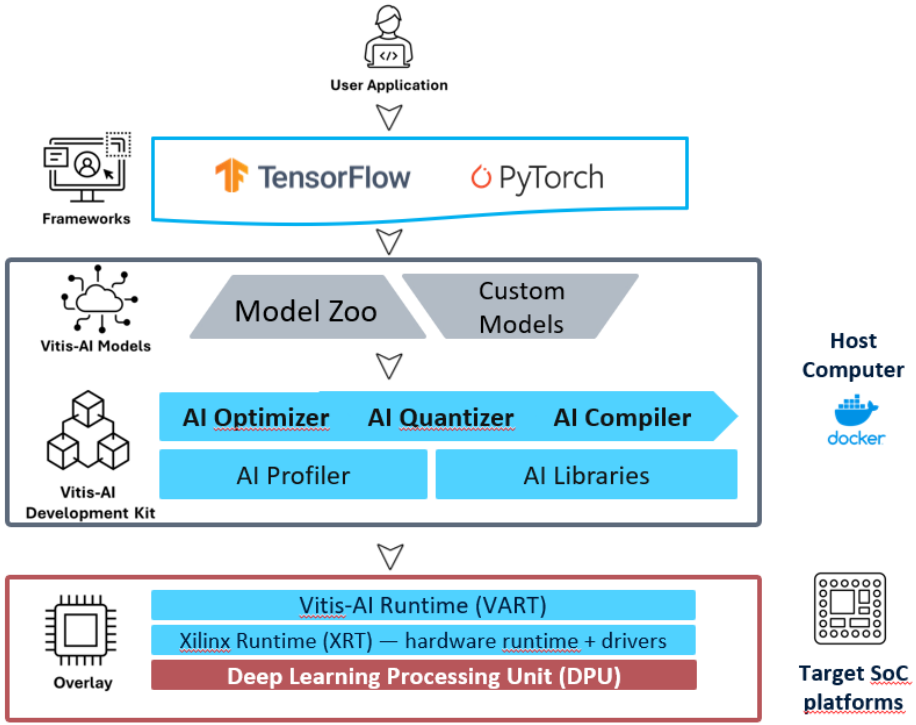
- **Limited Compute:** Edge devices lack high-performance CPUs/GPUs and have low clock speeds, making heavy DL models difficult to run in real time
- **Memory Constraints**: RAM and storage capacity are significantly reduced, restricting model size and intermediate RAM usage
- **Power Limitations**: Battery-powered or low-power embedded systems must balance performance with strict energy budgets

Two Key Ingredients for Edge AI Deployment

- Optimization techniques to reduce model complexity → TinyML approaches
- Specialized compute units → Hardware Accelerators

From DL models to FPGAs: The Xilinx's solution

- Xilinx toolchain: end-to-end ecosystem (tools, libraries) that Xilinx provides to bring DL models from high-level frameworks to SoC platforms (CPU+FPGA)



Three Main Components

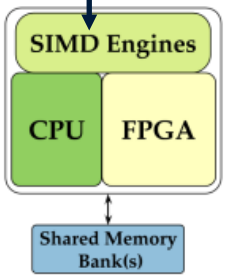
- **Vitis AI**: Set of software tools and libraries for model **optimization** (reducing computational complexity and number of operations) and **compilation** into a format executable by the hardware
- **Drivers** (VART): Software layer running on the embedded device that allows the optimized and compiled model to be executed on the hardware
- **DPU**: Xilinx Intellectual Property (IP) core. Customizable hardware accelerator implemented in the FPGA fabric, highly optimized for running CNN models

The Deep Learning Processing Unit (DPU)

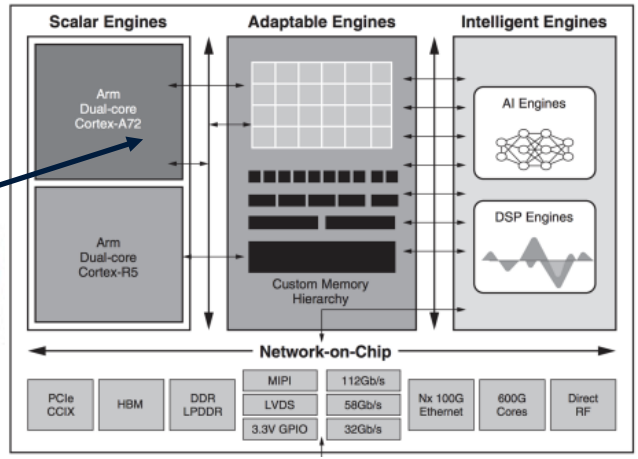
AMD-Xilinx FPGA Available in the Market

Standalone FPGA

ZYNQ technology

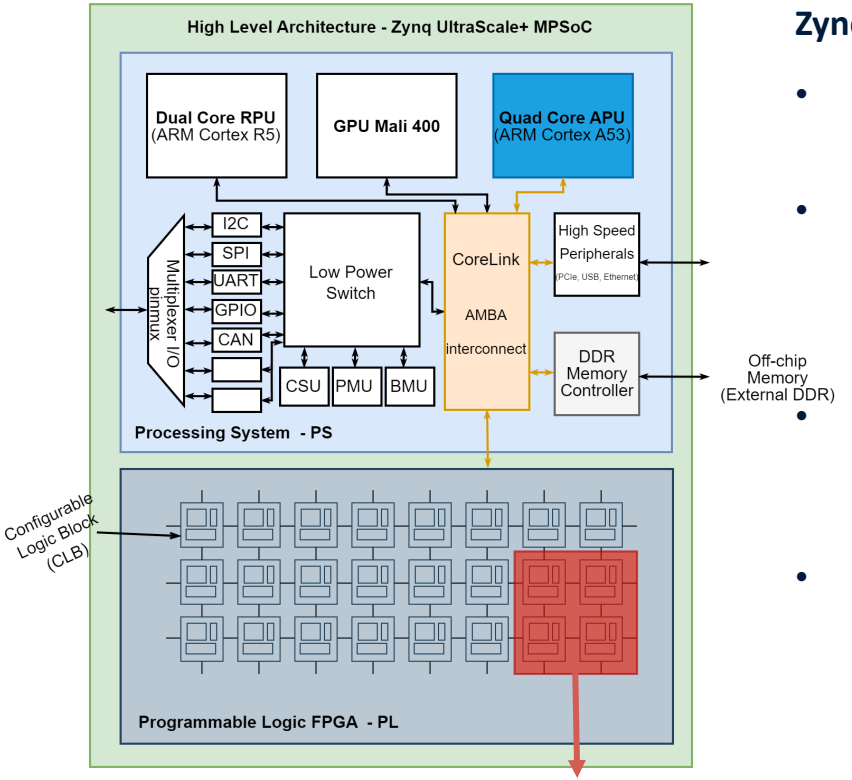


Versal Adaptive SoCs



MultiProcessor System-on-Chip (MPSoC)

DPU architecture for Zynq UltraScale+ MPSoC

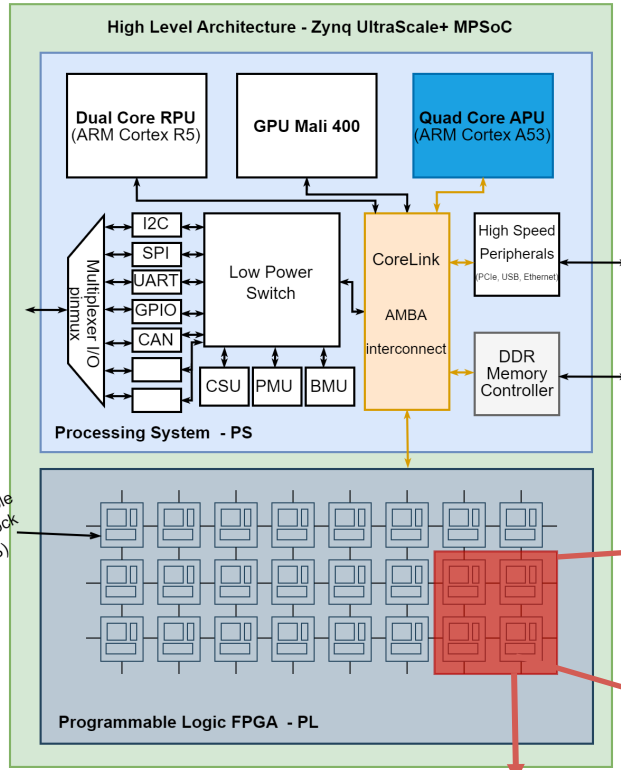


Zynq UltraScale+ MPSoC Overview

- Processing system (PS) and programmable logic (FPGA) in the same chip (system-on-a-chip).
- The PS integrates a quad-core **Application Processing Unit** (APU) based on 64-bit ARM Cortex-A53 processors. This APU handles high-speed general-purpose processing and provides significant computational capability.
- The PS also integrates a dual-core **Real-Time Processing Unit** (RPU) based on ARM Cortex-R5 processors, which is dedicated to executing real-time control tasks.
- In addition, the SoC can include a Mali-400 GPU, which provides graphics acceleration.

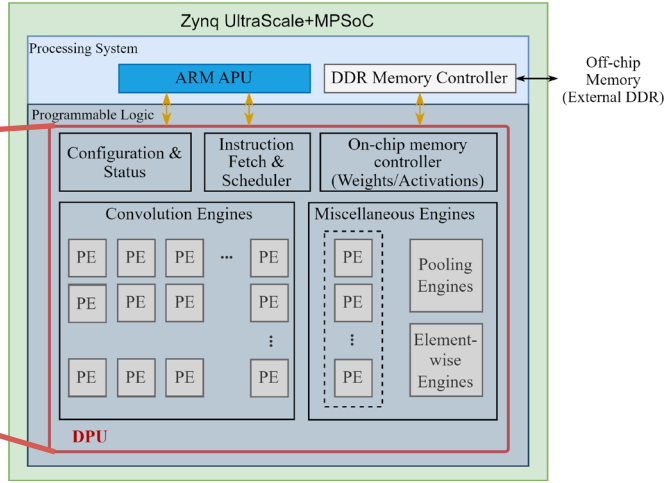
Soft IP core implemented in FPGA fabric - DPU

DPU architecture for Zynq UltraScale+ MPSoC



The **DPU: highly parallel hardware architecture optimized for CNN inference**

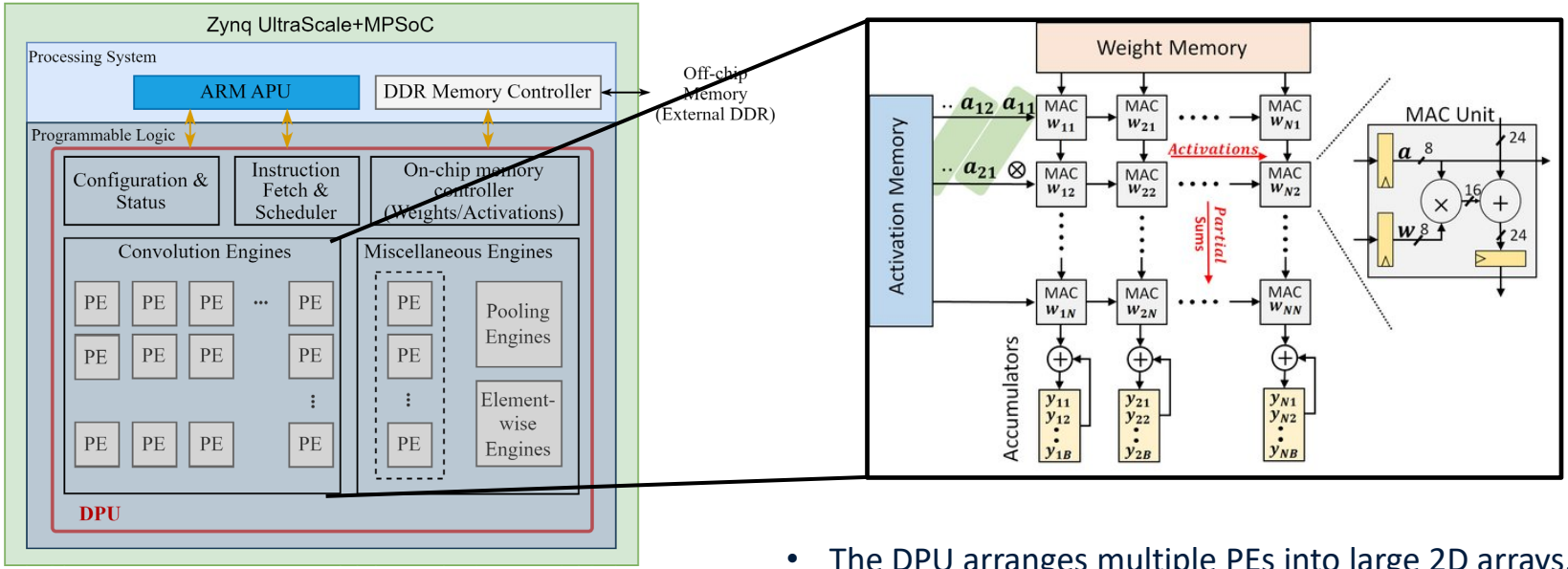
- **Soft IP core:** it is not a fixed block on the silicon, but implemented using the FPGA fabric (DSP slices, LUTs, flip-flops, and on-chip RAM)
- **Matrix of heterogeneous processing engines** (PEs) specialized in the different CNN operations (convolution, pooling, elementwise multiplication..)



Soft IP core implemented in FPGA fabric - DPU

DPU architecture for Zynq UltraScale+ MPSoC

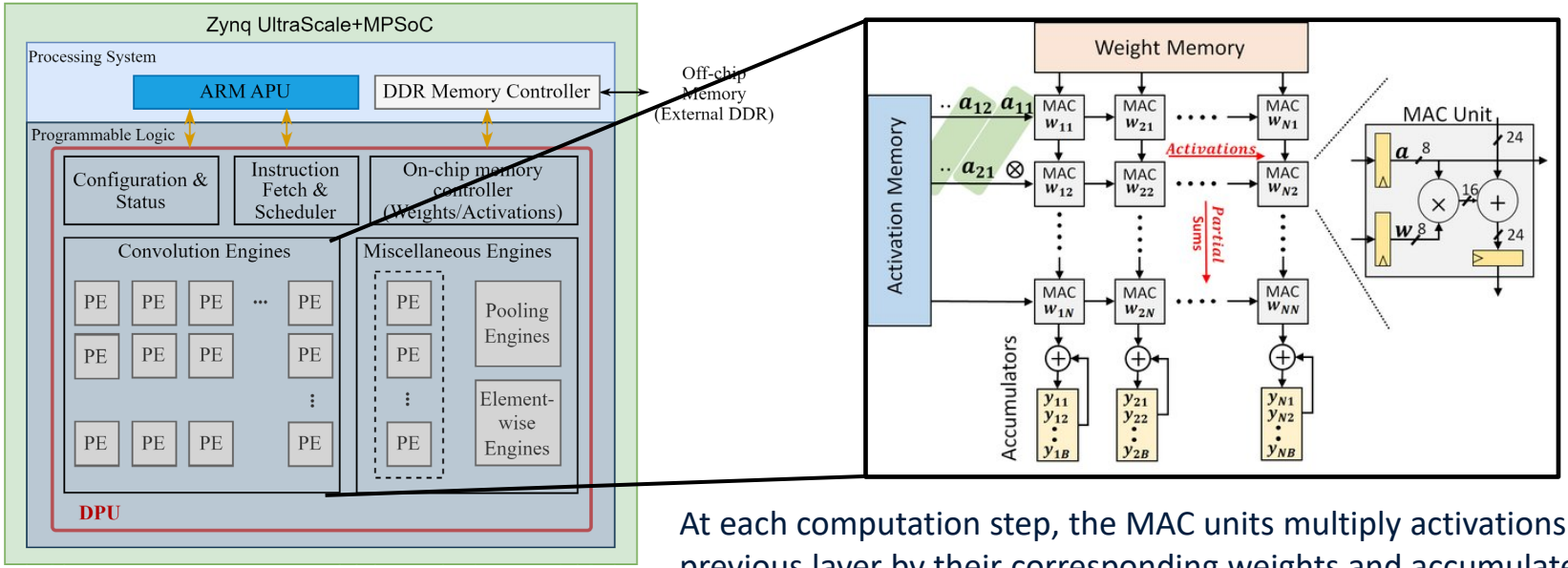
How are CNNs executed at the hardware level?



- The DPU arranges multiple PEs into large 2D arrays.
- The PEs take full advantage of the fine-grained building blocks, such as multipliers, adders, and accumulators in the FPGA fabric.
- Each PE includes a **MAC (Multiply-Accumulate) unit**.

DPU architecture for Zynq UltraScale+ MPSoC

How are CNNs executed at the hardware level?



At each computation step, the MAC units multiply activations from the previous layer by their corresponding weights and accumulate partial sums. Those partial sums are then combined to produce the output activations of a convolutional layer.

CNNs = the DPU can perform up to Thousands of MACs computed in parallel

DPU Features

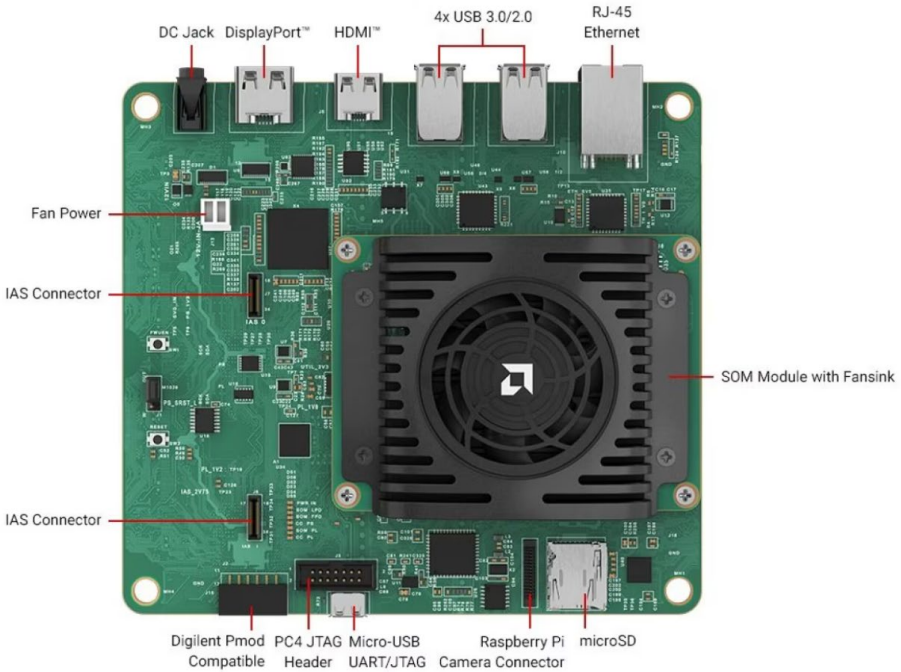
- The DPU is a configurable computation engine with different hardware architecture core sizes
- It includes a set of highly optimized instructions, and supports most convolutional neural networks, such as VGG, ResNet, GoogLeNet, YOLO, SSD, MobileNet, and others.
- The DPU requires instructions to implement a NN and accessible memory locations for input images as well as temporary and output data. A program running on the APU is also required to service interrupts and coordinate data transfers

Convolution Architecture	Pixel Parallelism (PP)	Input Channel Parallelism (ICP)	Output Channel Parallelism (OCP)	Peak Ops (operations/per clock)
B512	4	8	8	512
B800	4	10	10	800
B1024	8	8	8	1024
B1152	4	12	12	1150
B1600	8	10	10	1600
B2304	8	12	12	2304
B3136	8	14	14	3136
B4096	8	16	16	4096

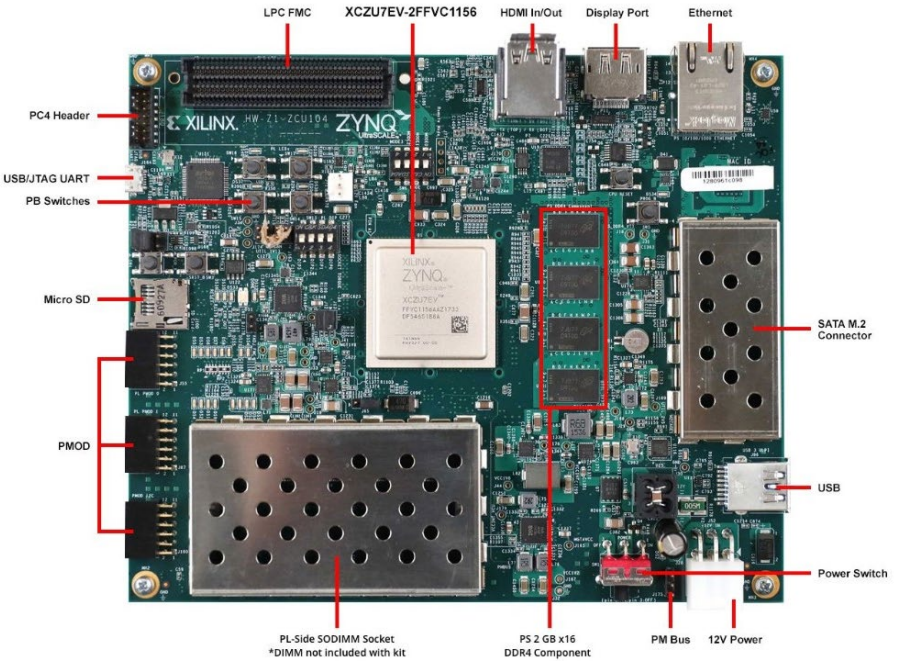
In each clock cycle, the convolution array performs a multiplication and an accumulation, which are counted as two operations. Thus, the peak number of operations per cycle is equal to PP*ICP*OCP*2.

Accelerator Cards: Kria KV260 and ZCU104

Kria KV260 Vision AI Starter Kit



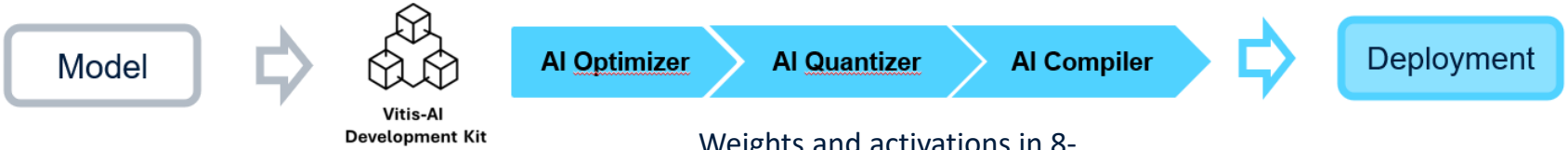
MPSoC ZCU104 Evaluation Kit



The Vitis AI development flow

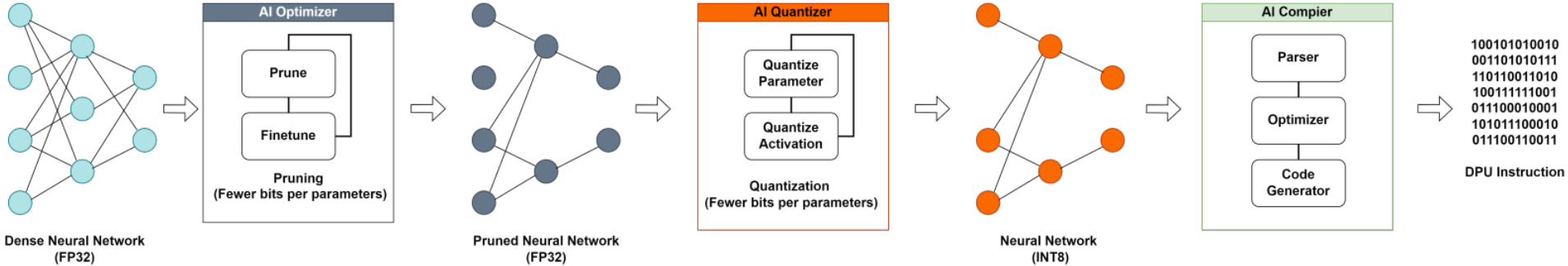
Vitis AI: Model Optimization and Deployment toolkit

General overview



Weights and intermediate activations in floating point 32-bit (FP32) precision

Weights and activations in 8-bit integer (INT8) precision

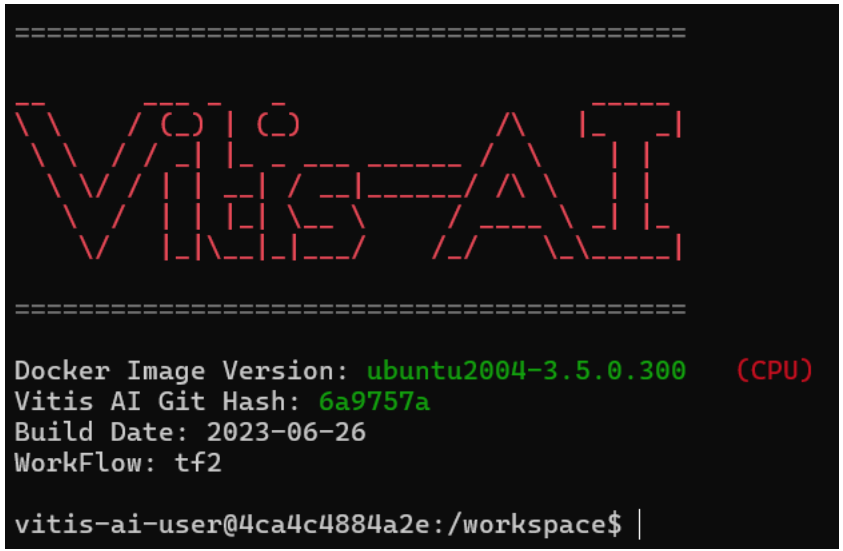


- Removes redundant connections and parameters through ***pruning***
- Converts the original FP32 network into an INT8 model
- Compiles the model into a hardware-friendly format: an instruction sequence optimized for DPU operations (**.xmodel**)

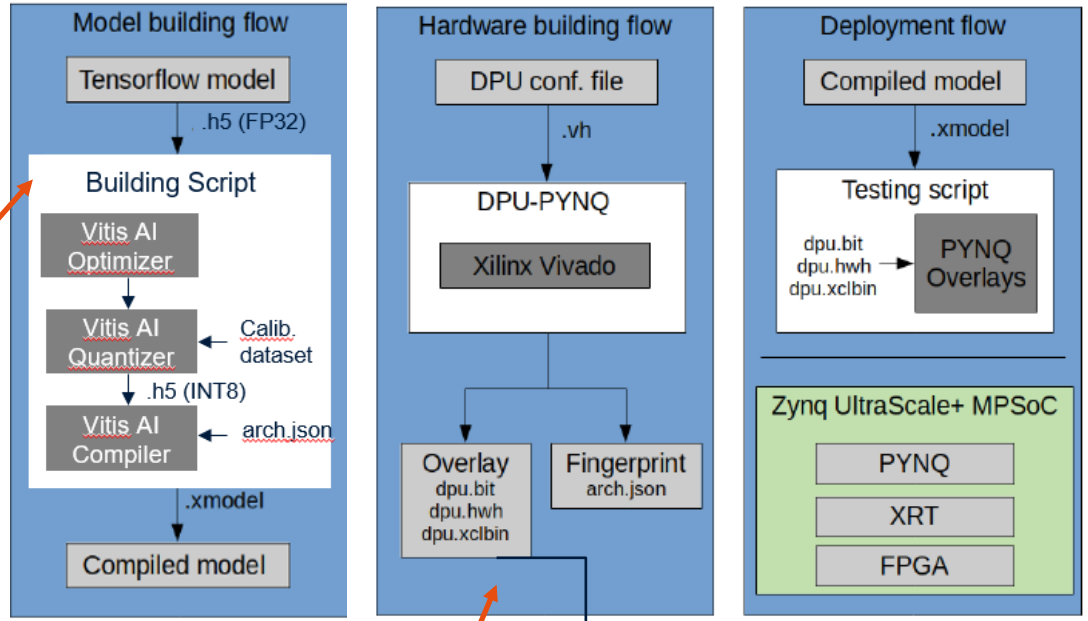
Setting up – Docker and Vitis-AI Installation

- **Vitis AI Docker-based installation**
- The Docker containers come in three main variants:
  - **CPU-only**
  - **GPU-enabled** (NVIDIA GPUs)
  - **ROCm-enabled** (AMD GPUs)
- Supported deep learning framework:
  - TensorFlow 1.x
  - TensorFlow 2.x
  - PyTorch

- We use the **CPU-only TensorFlow 2.x container**
- Check instructions at **Tutorial 0 – Vitis AI Setup**



Xilinx: proposed end-to-end deployment flow



Vitis AI software stack

FPGA design flow (Vivado / DPU IP integration)

Files defining the DPU hardware

Runtime execution on the target device

Software stack:

- VART (Vitis AI Runtime)
- XRT (Xilinx Runtime)
- PYNQ / Python APIs

What is Quantization?

- Quantization is a model optimization technique that reduces the numerical precision used to represent weights and activations in deep learning models. Its primary benefits include:
  1. **Model Compression** - lowers memory usage and storage.
  2. **Inference Acceleration** - speeds up inference and reduces energy consumption.
- While quantization is most often used for deployment on edge devices (e.g., phones, embedded hardware), it can also reduce infrastructure costs for large-scale inference in the cloud

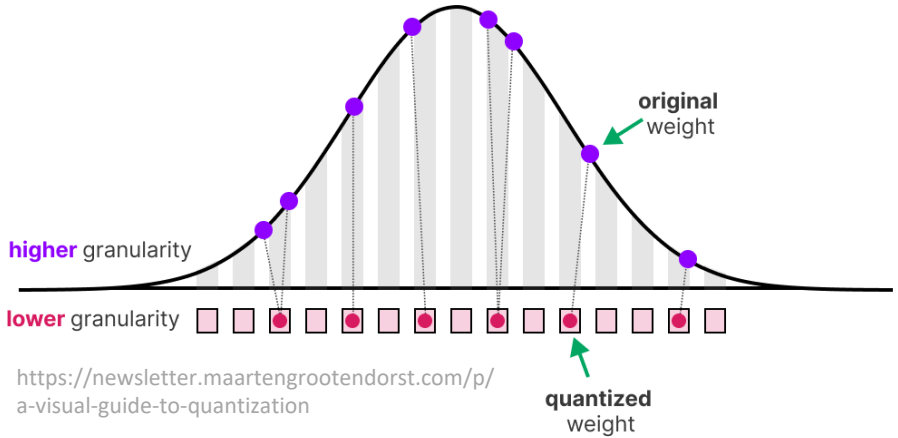
What is Quantization?

- Most DL models are trained in **floating-point 32-bit** (**FP32**) arithmetic to take advantage of a *wider dynamic range*. However, at inference, these models may take a longer time to predict results compared to reduced precision inference, causing some delay in the real-time responses, and affecting the user experience. For deployment in *embedded systems*, quantization is fundamental.

Quantization aims to reduce the precision of a model's parameter from higher bit-widths (like FP32) to lower bit-widths (like **8-bit integers, IN8**).

Why is quantization required on edge hardware?
FP32 → it allocate **4 bytes (32 bits)** to store each value
INT8 → it allocate **1 bytes (8 bits)** to store each value

Many hardware accelerators do not support FP32 inference directly. E.g., the DPU performs computations using INT8. In this case, quantization is a mandatory step.



Why do integers and floating-point numbers exist?

- **Integers** (signed/unsigned) represent numbers without a fractional part: …, −2, −1, 0, 1, 2, …
- **Floating-point** (single or double precision) approximate **real** numbers with a **fractional part**: 3.14, 0.1, −2.5, …

- Computers use a fixed number of bits to represent any piece of data. A bit string made up of n bits can represent up to 2^n distinct numbers. With limited bits, you cannot have both **huge range** and **infinite precision** at the same time. You must trade one for the other!

Why do integers and floating-point numbers exist?

Integers

- Represent whole numbers exactly (no rounding).
- All bits are used for the value: **perfect precision**.
- Limited range: errors happen only if **overflow** occurs (value exceeds the maximum/minimum).
- Example: 8-bit signed range −128 to 127
  - 100 + 20 = 120 (ok)
  - 120 + 20 = overflow (140 cannot be represented in this range)

Floating-point

- Represent real numbers with fractional parts.
- Bits are split between **precision (mantissa)** and **range (exponent)**.
- Can represent very large/small numbers, but with **rounding errors**.
- Example: 0.1 + 0.2 = 0.30000000000000004 (in float)

Floating point numbers

- Floating-point numbers are positive or negative numbers with a decimal point which are meant to represent real numbers.
- A floating-point number can be indicated with this generic notation:

$$(-1)^s \times m \times b^e$$

- $s \in \{0,1\}$ is the **sign** bit (0 = positive, 1 = negative)
- m is the **mantissa** (also called *significand*) whose length determines the precision to which numbers can be represented
- b is the **base** (also called *radix*)
- e is the **exponent**

A simple example in base 10

- Scientific notation: -2.32×10^{-2}

Sign bit = 1

Mantissa
a *normalized* number of certain precision

Exponent
scale factor to determine the position of the decimal point (the **base** is implied)

- NB: floating point numbers can have multiple forms:
  $-2.32 \times 10^{-2} = -23.2 \times 10^{-3} = -232 \times 10^{-4} = -2320 \times 10^{-5}$

But.. it is desirable for each number to have a unique representation → *Normalized Form*
We normalize **mantissa** in the range [1, b-1], where b is the base, e.g.:
[1, 9] for DECIMAL → e.g., 1.18, 9.28
1 for BINARY → e.g., 1.001

IEEE 754 Floating-Point Standard

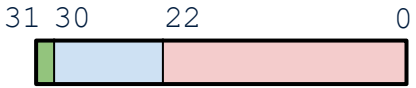
- **IEEE 754** is the most widely used standard for floating-point arithmetic in computers and defines how floating-point numbers are represented in binary → the format (sign, exponent, mantissa)

IEEE 754 introduces the following constraints:

- *Binary base* $\boldsymbol{b = 2}$
- *Normalized mantissa* or *fraction* $\boldsymbol{m = 1.f}$ (there are always an implicit leading **1**)
- *Biased exponent* $\boldsymbol{E = e + bias}$
- The number is encoded (stored) into three parts: s, E, and f

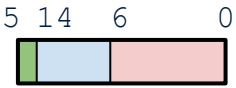
$$\boldsymbol{value = (-1)^s \times (1.f) \times 2^{E-bias}}$$

IEEE 754 32-bit (FP32)
Single precision
float32
binary32



E=8, **f**=23

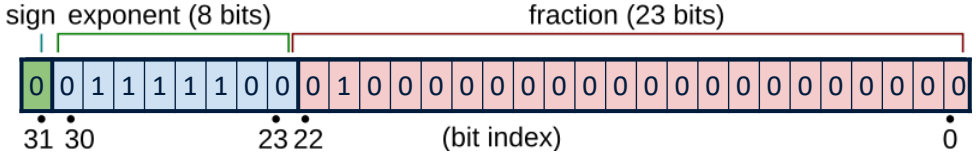
IEEE 754 16-bit (FP16)
Half precision
float16
binary16



E=5, **f**=10

Sign field Biased exponent (**e**) Fraction (**f**)

Exercise: converting decimal to FP32



$$value = (-1)^s \times (1.f) \times 2^{E-127}$$

- Sign
- Biased Exponent E (8-bit unsigned integer from 0 to 255) with bias=127
- 23-bit fraction f

$$\begin{aligned} 1.f &= (1.b_{22}b_{21}b_{20}\ldots b_0)_2 \\ &= b_{22} \times 2^{-1} + b_{21} \times 2^{-2} + b_{20} \times 2^{-3} + \cdots \\ &= 1 + \sum_{i=1}^{23} b_{23-i}2^{-i} \end{aligned}$$

Consider a real number with an integer and a fraction part

1) Convert the integer part to binary (*divide-by-2 method* or *sum of powers of two method*)
2) Convert the fraction part to binary (**multiply-by-2 method**)
3) Add the two results and adjust them to produce a proper final conversion (power-of-two normalized)
4) Encode biased exponent: $E = e + bias$

Exercise: converting decimal to FP32

Convert $\mathbf{12.375_{10}}$ to IEEE 754 binary32

1) Convert the integer part to binary:
$\mathbf{12_{10}} = 8 + 4 = 1 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 0 \times 2^0 = \mathbf{1100_2}$

2) Convert the fraction part to binary (multiply-by-2 method):
Repeatedly multiply by 2; **the integer bit each time is the next binary digit**. Repeat until a fraction of zero is found or until the precision limit is reached (maximum of 23 fraction digits)

- $0.375 \times 2 = \mathbf{0}.75 \rightarrow$ bit **0**
- $0.75 \times 2 = \mathbf{1}.5 \rightarrow$ bit **1**, keep fractional part 0.5
- $0.5 \times 2 = \mathbf{1} \rightarrow$ bit **1**, keep fractional part 0.0 (stop)

So: $\mathbf{0.375_{10}} = \mathbf{0.011_2}$

Exercise: converting decimal to FP32

Convert $\mathbf{12.375_{10}}$ to IEEE 754 binary32 $\quad \mathbf{12_{10} = 1100_2} \quad \mathbf{0.375_{10} = 0.011_2}$

3) Combine integer and fraction and normalize to form $(\mathbf{1}.\boldsymbol{f}) \times \mathbf{2}^{\boldsymbol{e}}$

$$12.375_{10} = 1100.0011_2 = 1.1000011_2 \times 2^3$$

So: sign bit $s = 0$ (positive)

Exponent: $e = 3$

only the fraction part of the mantissa f

4) Exponent encode (bias=127)

$$E = e + 127 = 130_{10} = 10000010_2$$

$$0\ 10000010\ 10000110000000000000000 = 0x41460000$$

sign bit s

Biased exponent E

only the fraction part of the mantissa f

Add zeros to reach 23 bits for the fraction part

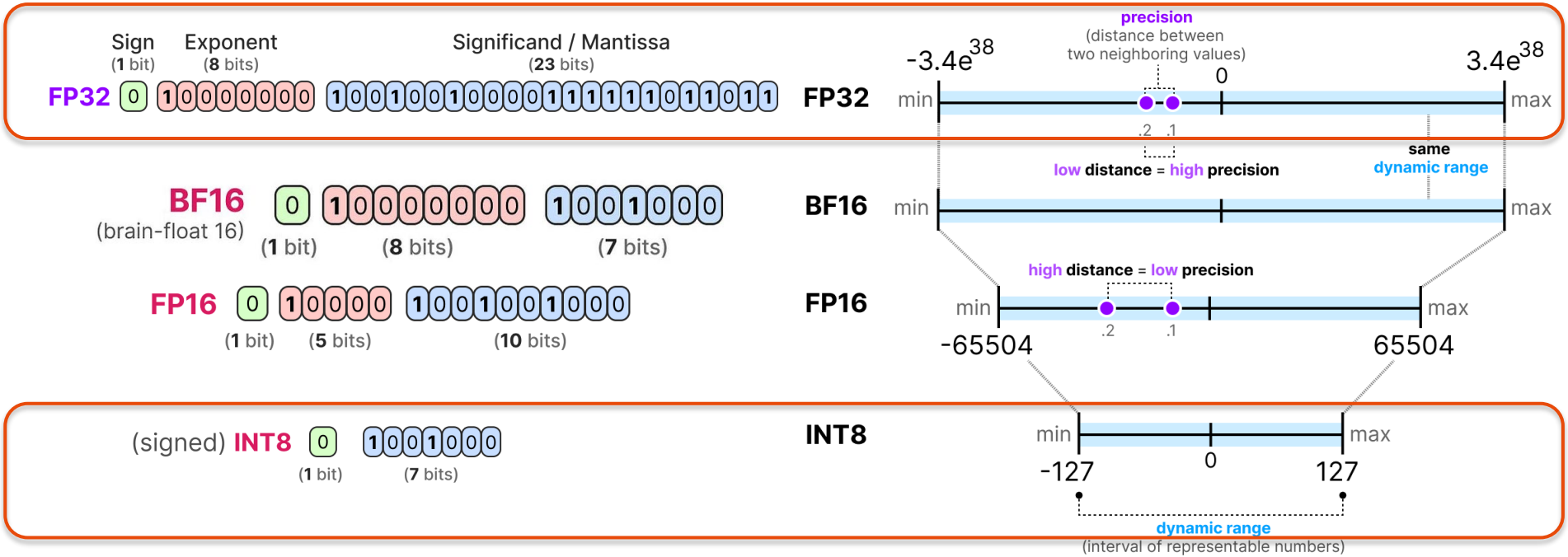
Types of quantization

Specified by:

- **Format**
  - based on the numerical representation used to encode weights and activations after quantization. E.g., FP32 → FP16, FP32 → INT8
- **Mapping type**
  - Uniform vs Non-Uniform Quantization
  - Symmetric vs Asymmetric Quantization
- **Granularity**
  - Layer-wise vs Channel-wise
- **Method**
  - Post-Training Quantization (PTQ) vs Quantization-Aware Training (QAT)

Recommended reading: Gholami *et al*., 2021, *A Survey of Quantization Methods for Efficient Neural Network Inference*

Format



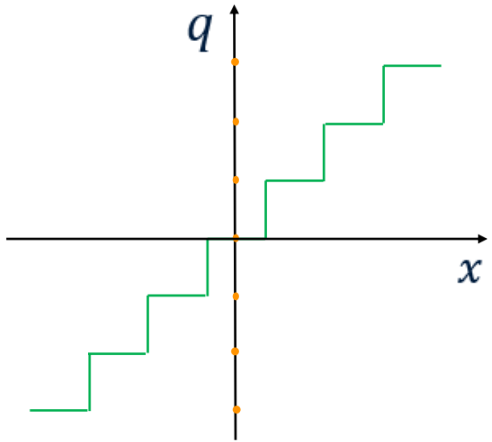
dynamic-range = interval of representable numbers

precision of the representation = distance between two neighboring representable numbers

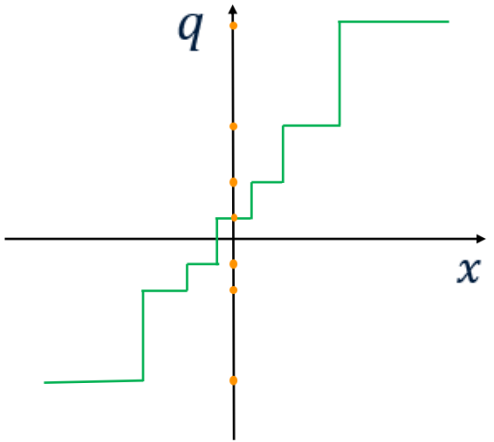
Mapping type: Uniform vs Non-Uniform quantization

Real values in the continuous domain x are mapped into discrete, lower precision values in the quantized domain q.

- Uniform quantization: **linear mapping** from floating point values to quantized integer values. Uniform quantization uses **equally spaced levels** between quantized values.
- **Non-uniform** quantization: non-linear mapping. Distances between quantized values can vary (**unequal spacing).**



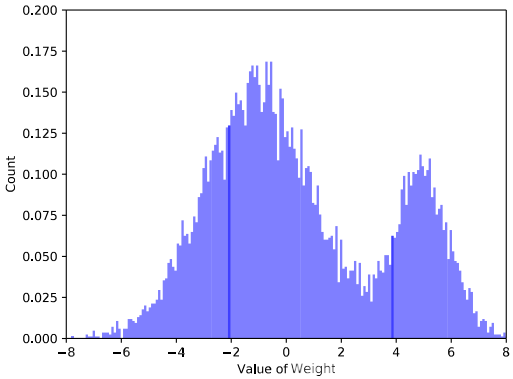
(a) Uniform quantization

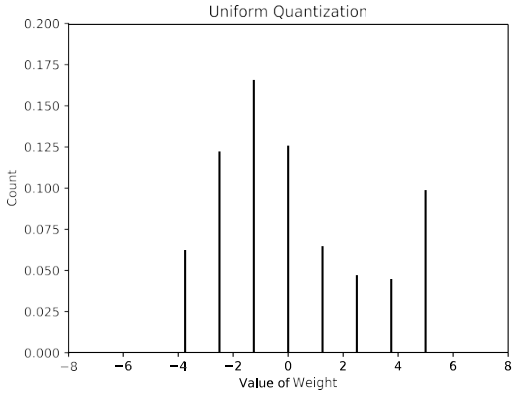


(a) Non-uniform quantization

Mapping type: Uniform vs Non-Uniform quantization

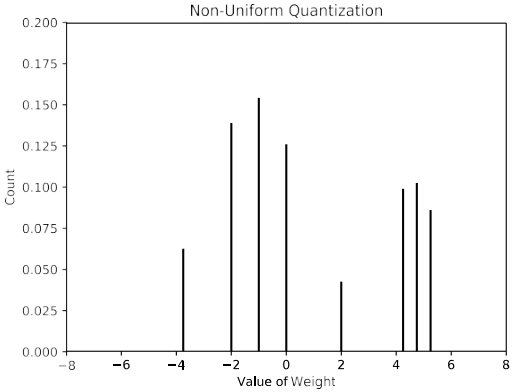
Typical distribution of neural network weights (often centered around zero).

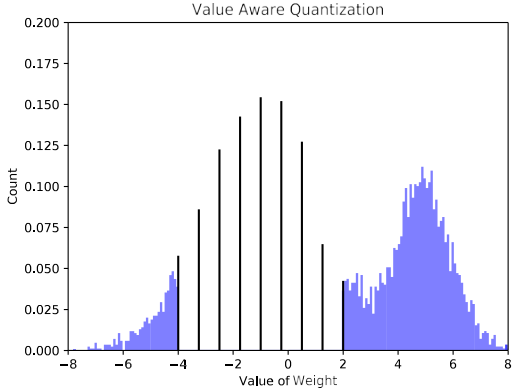




Uniform quantization: quantization levels are equally spaced

Non-uniform quantization: quantization levels are distributed according to the data distribution.





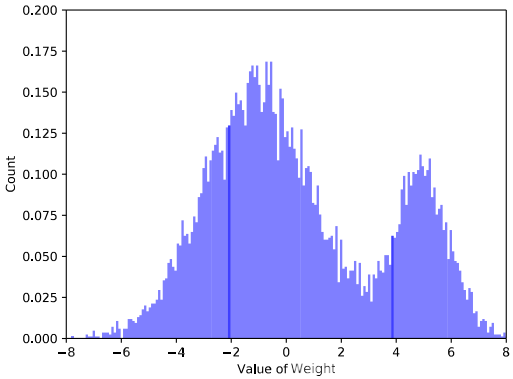
Value-aware quantization: more levels are allocated where values are more frequent.

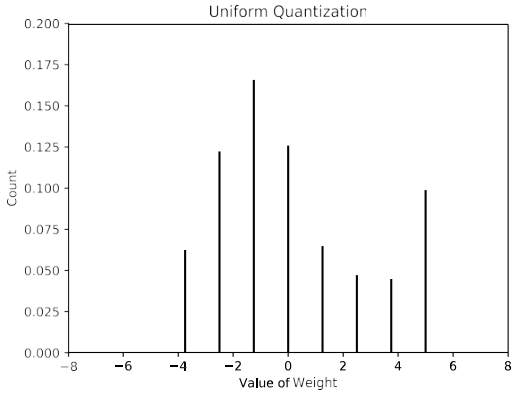
Maastricht University
Department of Advanced Computing Sciences

Park, Eunhyeok, Sungjoo Yoo, and Peter Vajda. "Value-aware quantization for training and inference of neural networks." ECCV, 2018

Mapping type: Uniform vs Non-Uniform quantization

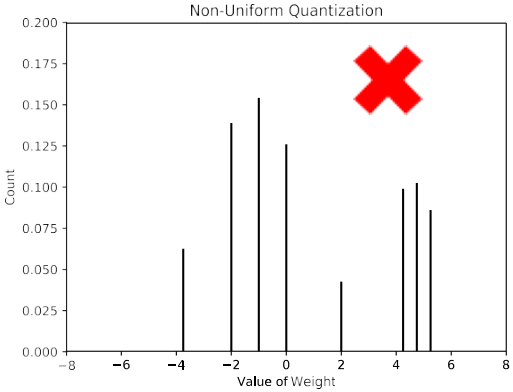
Typical distribution of neural network weights (often centered around zero).

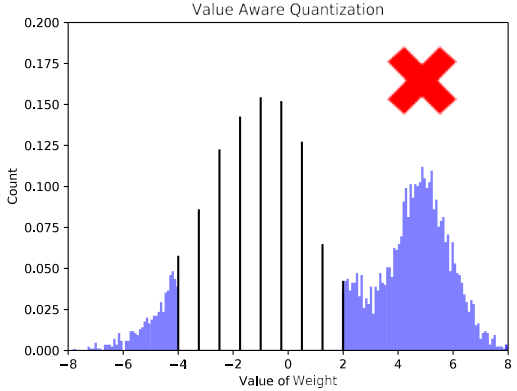




Uniform quantization: quantization levels are equally spaced

Non-uniform quantization: quantization levels are distributed according to the data distribution.





Value-aware quantization: more levels are allocated where values are more frequent.

Non-uniform quantization is not efficient for hardware deployment.

Maastricht University
Department of Advanced Computing Sciences

Park, Eunhyeok, Sungjoo Yoo, and Peter Vajda. "Value-aware quantization for training and inference of neural networks." ECCV, 2018

Uniform quantization

- Uniform quantization means that quantization levels are equally spaced. the constant distance between two consecutive quantized values is called the scale factor S: $\Delta = S$
- **Quantization**: a floating-point value x in the range $[x_{min}, x_{max}]$ is mapped to an integer value q in the range $[q_{min}, q_{max}]$ using a scale factor S and optionally a zero-point Z

$$q = round\left(\frac{x}{S} + Z\right),\ q = clip(q_{min}, q_{max}) \qquad S = \frac{\beta - \alpha}{q_{max} - q_{min}}$$

- **Dequantization**: reconstructs a real approximate value $\tilde{x} = S \times (q - Z)$

$$Z = round\left(q_{min} - \frac{\alpha}{S}\right)$$

where:

- x = original float (real)
- q = quantized integer
- S = scale factor (step size between quantized values)
- Z = zero-point or offset (integer representing real value 0)

Uniform quantization

- **Scale factor**: $S = \frac{\beta - \alpha}{q_{max} - q_{min}}$
- **Zero-point**: $Z = round\left(q_{min} - \frac{\alpha}{S}\right)$

where:

- $[\alpha, \beta]$ = clipping range, a bounded range that we are clipping the real values with
- $[q_{min}, q_{max}]$ = quantized integer range, i.e., the minimum and maximum representable values in the target integer format

$[q_{\min}, q_{\max}]$ depends on the quantization scheme

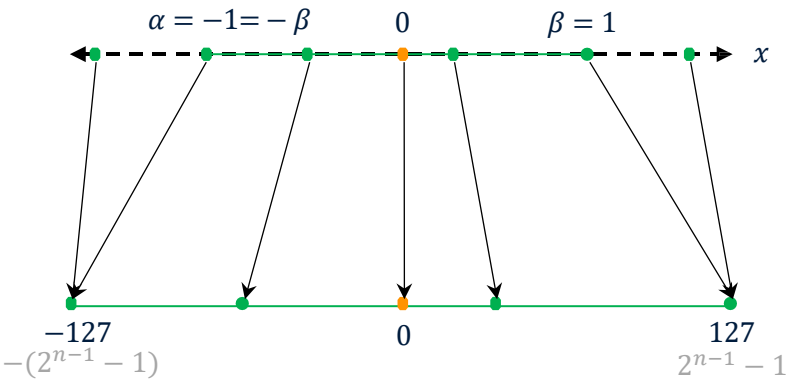
- *signed full range* $[-2^{n-1}, 2^{n-1} - 1]$, INT8: $[-128, 127]$
- *signed restricted range* $[-(2^{n-1}-1), 2^{n-1} - 1]$, INT8: $[-127, 127]$
- *unsigned* $[0, 2^n - 1]$, UINT8: $[0, 255]$

Uniform quantization: Symmetric vs Asymmetric

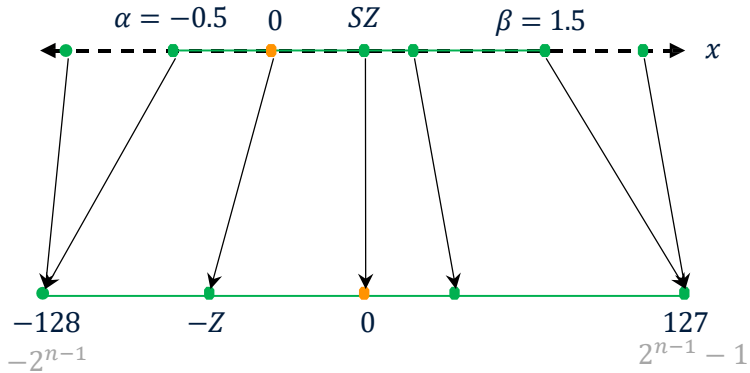
How do we choose the clipping range $[\alpha, \beta]$?

The choice of the clipping range is critical during quantization.

Determining these bounds is known as **calibration**, and it influences whether the quantization is **symmetric** or **asymmetric**.



(b) **Symmetric quantization** with restricted range map



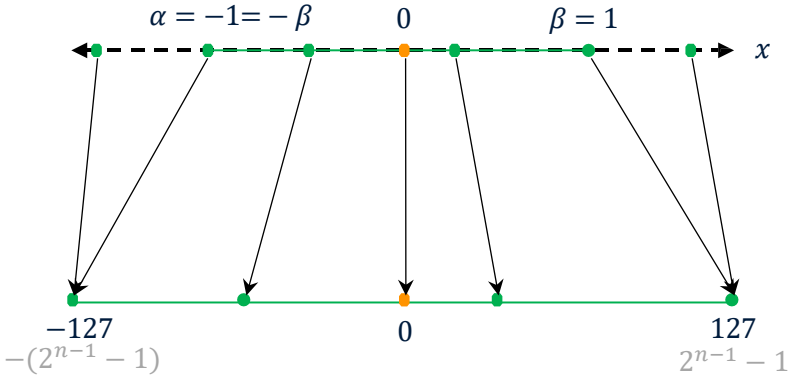
(a) **Asymmetric (or affine) quantization** with full range map

Uniform quantization: Symmetric

How do we choose the clipping range $[\alpha, \beta]$?

- Symmetric quantization maps the floating-point values *symmetrically around zero*. This means that a symmetric range is defined as the clipping range $\alpha = -\beta$. The floating-point value 0.0 is mapped exactly to the integer value 0.

- A common approach to determine clipping bounds is the **AbsMax method**. Given the floating-point value x in the range $[x_{min}, x_{max}]$, we set $\alpha = -\beta = \max(|x_{\min}|, |x_{\max}|)$



(b) **Symmetric quantization** with restricted range map

Zero offset $Z = 0$

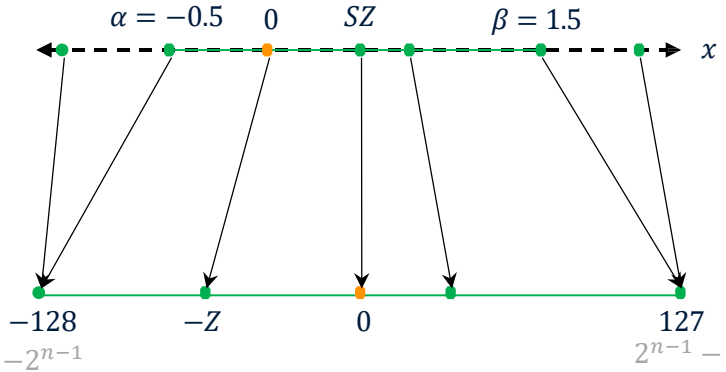
Scale factor $S = \frac{2\beta}{q_{max} - q_{min}}$

- *Restricted range* map
  $[q_{min}, q_{max}] = [-(2^{n-1}-1), 2^{n-1}-1] = [-127, 127]$
- *Full range* map
  $[q_{min}, q_{max}] = [-2^{n-1}, 2^{n-1}-1] = [-128, 127]$

Uniform quantization: Asymmetric

How do we choose the clipping range $[\alpha, \beta]$?

- In asymmetric quantization, there is no symmetry constraint between α and β
- A common approach to determine clipping bounds is the **Max-min method:** $\alpha = x_{min}, \beta = x_{max}$.
- It maps the exact min/max floating-point values observed in the tensor $[x_{min}, x_{max}]$ to the min/max values of the integer range $[q_{min}, q_{max}]$.



(a) **Asymmetric (or affine) quantization** with full range map

Non-zero offset $Z = round\left(q_{min} - \frac{x_{min}}{S}\right)$

Scale factor $S = \frac{x_{max} - x_{min}}{q_{max} - q_{min}}$

$[q_{min}, q_{max}]$ = full integer range

- [0,255] for unsigned INT8
- [−128,127] for signed INT8

Symmetric quantization: exercise

- Quantize the following FP32 vector using **signed INT8 symmetric quantization** with range $[-127, 127]$: $x = [-7.59, -4.57, -1.95, 0.00, 3.02, 3.08, 10.8]$. Calculate the quantization error.

Symmetric quantization: exercise

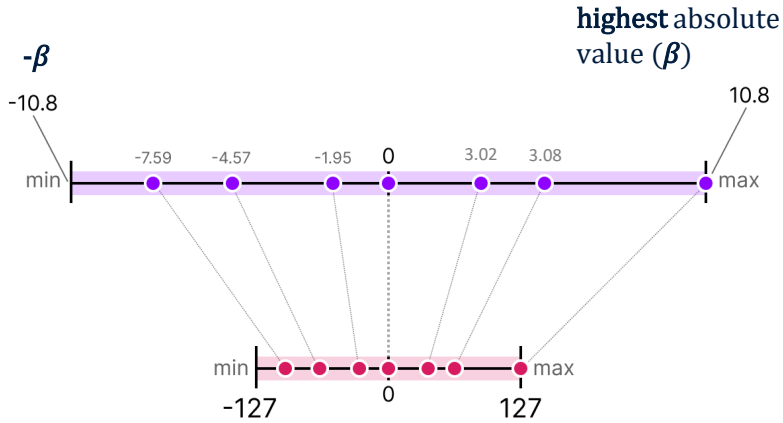
- Quantize the following FP32 vector using **signed INT8 symmetric quantization** with range $[-127, 127]$: $x = [-7.59, -4.57, -1.95, 0.00, 3.02, 3.08, 10.8]$. Compute the quantization error.

1) Determine the absolute maximum value
   $\beta = \max(|x_{min}|, |x_{max}|) = 10.8$

2) Compute the scale factor (using absmax symetric quantization)
   $S = \frac{2\beta}{q_{max} - q_{min}} = \frac{2\beta}{127-(-127)} = \frac{\beta}{127} = 0.085$

3) Quantize the vector (NB: $Z = 0$) (round to nearest)
   $q_i = round\left(\frac{x_i}{S}\right)$ with $q = clip(q_{min}, q_{max})$

$$q = [-89, -54, -23, 0, 36, 36, 127]$$



Symmetric quantization: exercise

- Quantize the following FP32 vector using **signed INT8 symmetric quantization** with range $[-127, 127]$: $x = [-7.59, -4.57, -1.95, 0.00, 3.02, 3.08, 10.8]$. Compute the quantization error.

$$q = [-89, -54, -23, 0, 36, 36, 127]$$

4) Compute the quantization error

- Dequantize: recover the floating-point approssimation $\tilde{x_i} = S \times q_i$
- Quantization error: compute the absolute error (difference) for each element $e_i = |x_i - \tilde{x_i}|$

$$\tilde{x} = [-7.57, -4.59, -1.96, 0, 3.06, 3.06, 10.80]$$

$$e = [0.03, 0.02, 0.01, 0, 0.04, 0.02, 0]$$

Symmetric quantization: exercise

- Quantize the following FP32 vector using **signed INT8 symmetric quantization** with range $[-127, 127]$: $x = [-7.59, -4.57, -1.95, 0.00, \mathbf{3.02}, \mathbf{3.08}, 10.8]$. Compute the quantization error.

$$q = [-89, -54, -23, 0, \mathbf{36}, \mathbf{36}, 127]$$

4) Compute the quantization error

- Dequantize: recover the floating-point approssimation $\tilde{x_i} = S \times q_i$
- Quantization error: compute the absolute error for each element $e_i = |x_i - \tilde{x_i}|$

$$\tilde{x} = [-7.57, -4.59, -1.96, 0, \mathbf{3.06}, \mathbf{3.06}, 10.80]$$

$$e = [0.03, 0.02, 0.01, 0, 0.04, 0.02, 0]$$

You can see certain values, such as **3.08** and **3.02** being assigned to the INT8, namely **36**. When you dequantize the values to return to FP32, they lose some precision and are not distinguishable anymore.

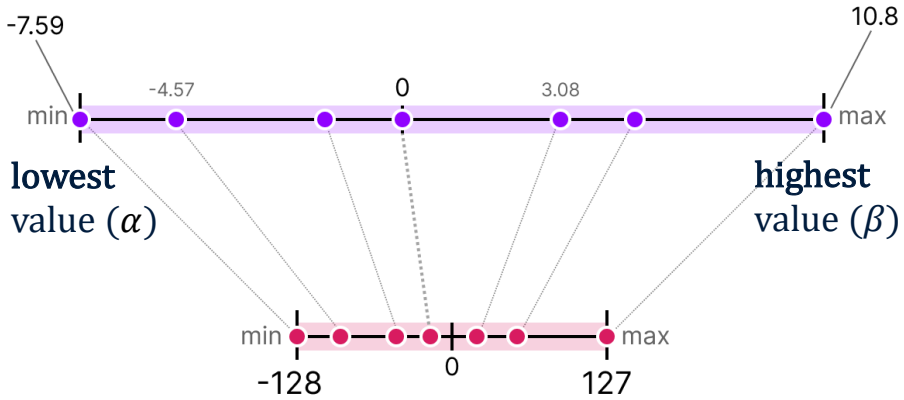
Asymmetric quantization: exercise

- Quantize the same FP32 vector $x = [-7.59, -4.57, 3.08, 10.8]$ using signed INT8 **asymmetric** quantization (range $[-128, 127]$).

Asymmetric quantization: exercise

- Quantize the same FP32 vector $x = [-7.59, -4.57, 3.08, 10.8]$ using signed INT8 **asymmetric** quantization (range $[-128, 127]$).

1) Use the max-min method ($\alpha = x_{min}, \beta = x_{max}$) to calculate the
- Scale factor $S = \frac{x_{max} - x_{min}}{q_{max} - q_{min}}$
- Offset $Z = round\left(q_{min} - \frac{x_{min}}{S}\right)$



In this case:

$$S = \frac{x_{max} - x_{min}}{q_{max} - q_{min}} = \frac{10.8 - (-7.59)}{127 - (-128)} = \frac{18.39}{255} = 0.072$$

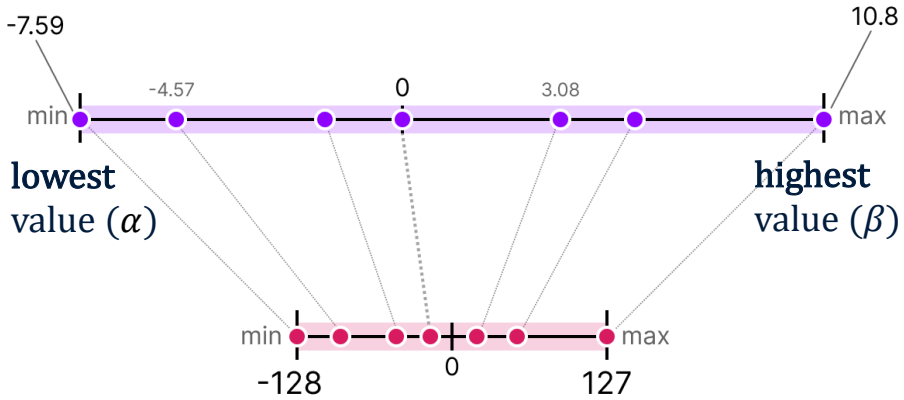
$$Z = round\left(q_{min} - \frac{x_{min}}{S}\right) = round(-22.7553) = -23$$

Asymmetric quantization: exercise

- Quantize the same FP32 vector $x = [-7.59, -4.57, 3.08, 10.8]$ using signed INT8 **asymmetric** quantization (range $[-128, 127]$).

2) Quantize using $q_i = round\left(\frac{x_i}{S} + Z\right)$
with $q_i = clip(q_{min}, q_{max})$

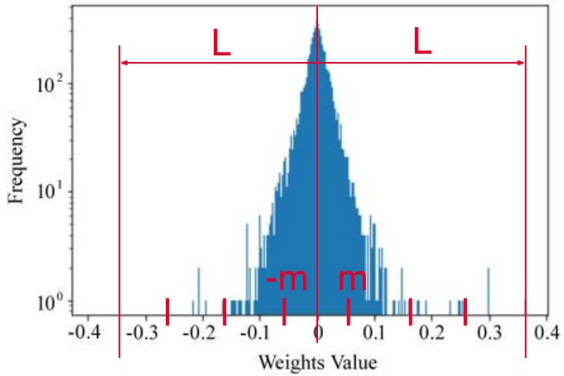
$$q = [-128, -86, 20, 127]$$



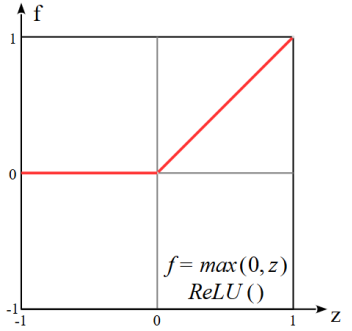
Asymmetric vs Symmetric Quantization in NNs

Feature	Symmetric	Asymmetric
FP32 range	Maps $[-\beta, \beta]$ with $\max(\lvert x_{\min}\rvert, \lvert x_{\max}\rvert)$	Maps $[x_{min}, x_{max}]$
INT8 range	signed integer range, [−127, 127] or [−128, 127]	[0,255] for unsigned or [−128, 127] for signed
Zero mapping	$0.0 \rightarrow 0$	$0.0 \rightarrow Z$ (integer zero-point)
Zero-point	$Z = 0$	$Z \neq 0$
Parameters	Scale (S)	Scale (S) and Zero-point (Z)
Best for **(Just general guidelines!)**	Distributions centered around zero (e.g, weights)	Skewed distributions (e.g., activations after ReLU)

The weight distribution follows a gaussian-like distribution



Activation values, especially after functions like ReLU, often have highly asymmetric distributions (e.g., always non-negative)



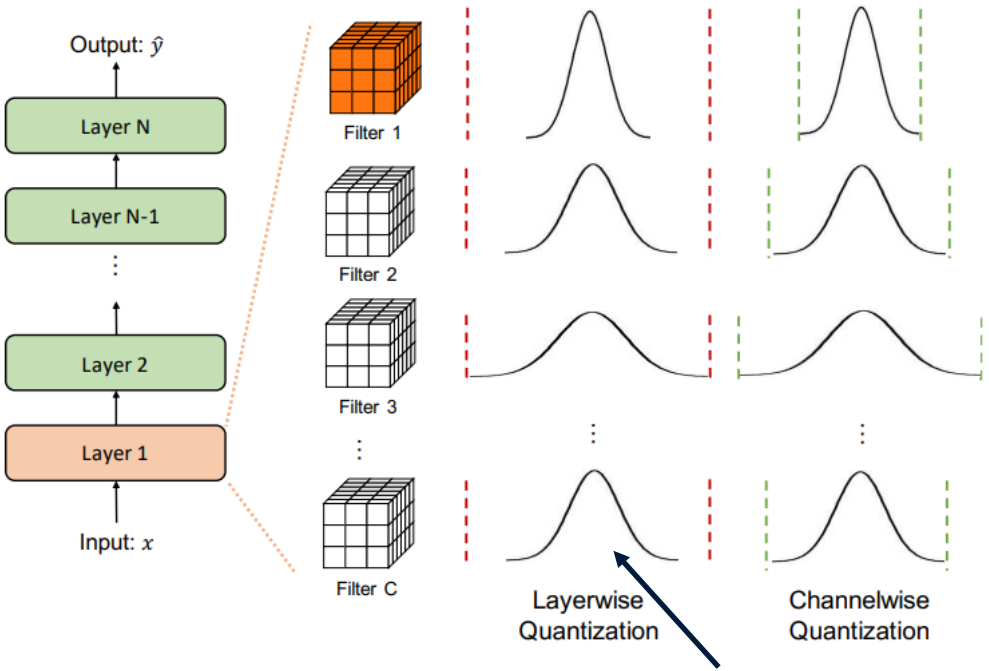
Quantization granularity: Layer-wise vs Channel-wise

Layer-wise quantization:

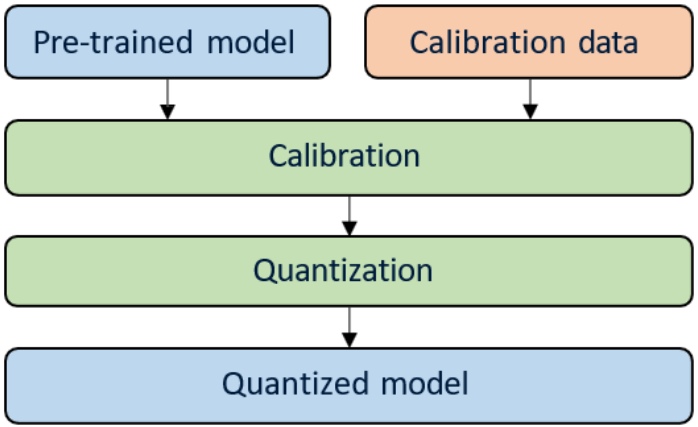
- Use the same clipping range $[\beta, \alpha]$ for all the convolution kernels in a convolutional layer
- One examines the statistics of the entire parameters in that layer, and then uses the same clipping
- range for all the convolutional filters
- Usually sub-optimal since the weights in a layer can have different ranges

Channel-wise Quantization:

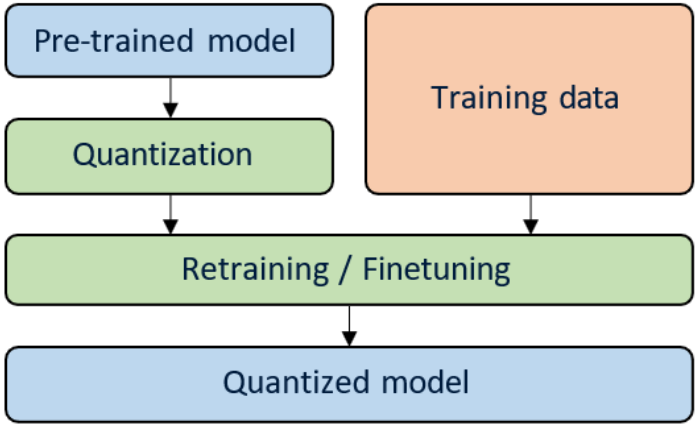
- Use different clipping ranges $[\beta, \alpha]$ for each convolutional filter
- Usually results in better accuracy as the range can be better captured for each output channel
- Has negligible overhead



Quantization method: PTQ vs QAT



Post-Training Quantization (PTQ)



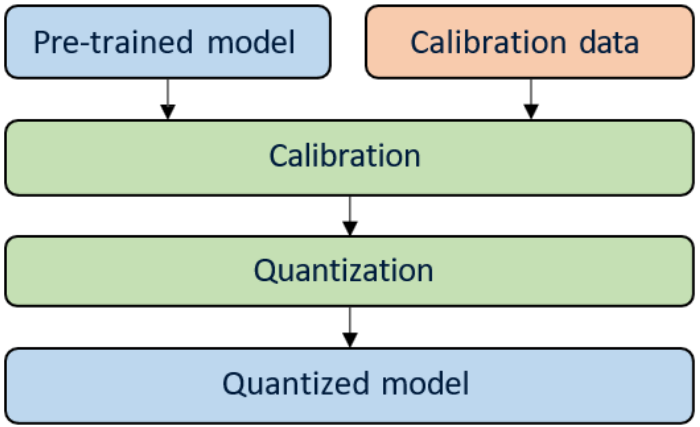
Quantization-Aware Training (QAT)

PTQ

PTQ converts a trained FP32 model into an INT8 model without retraining the network.

The process works as follows:

1. A small *calibration dataset* is passed through the network.
2. The quantizer observes the range of activations and weights in each layer.
3. Based on these statistics, the quantizer determines the scaling factors needed to map FP32 values to INT8.



PTQ

Advantage. This approach is very fast and simple because it does not require additional training.

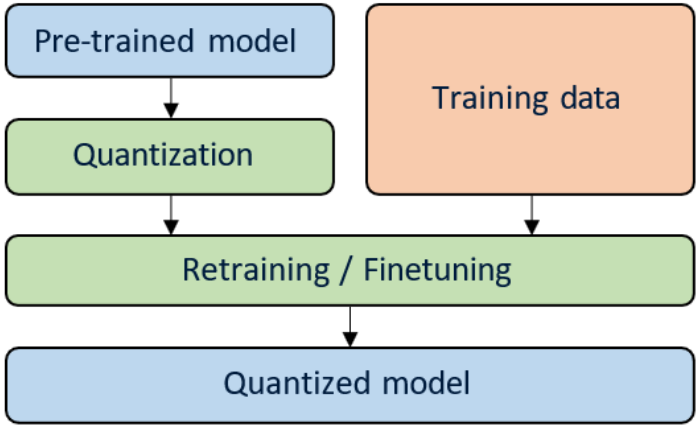
Disadvantage. For more complex networks, the model accuracy may decrease slightly after quantization

QAT

QAT simulates the effects of quantization during the training process.

During training, fake quantization operators are inserted in the network to simulate INT8 inference. This means that during training:

- weights and activations are quantized and immediately dequantized, simulating INT8 precision
- all computations and gradients (backpropagation) remain in FP32
- The forward pass experiences quantization error (rounding, clipping, loss of precision)
- The network learns to compensate for quantization errors



QAT

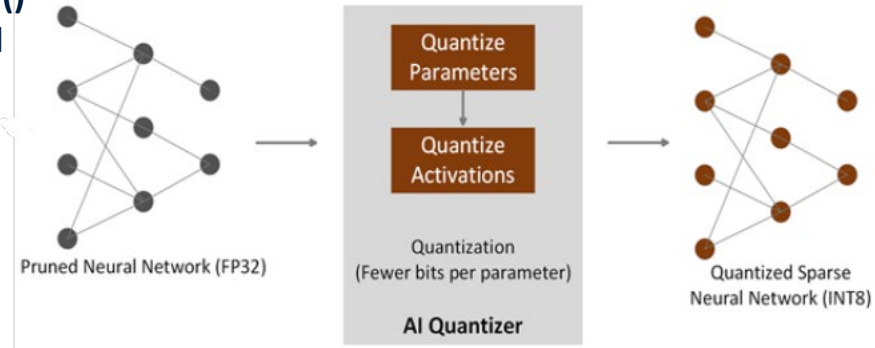
Advantage. The model can adapt to the quantization constraints, which usually results in higher accuracy

Disadvantage. It requires additional training.

How quantization works in Vitis AI – AI Quantizer

The Python class **quantizer = vitis_quantize.VitisQuantizer()** is the main interface provided by Vitis AI to perform model quantization in TensorFlow/Keras.

It takes a pre-trained FP32 model and converts it into a quantized INT8 model that can run efficiently on the DPU accelerator.



The quantizer performs several steps automatically:

1. Graph analysis. The neural network graph is analyzed layer by layer to determine which operations can be quantized.
2. Insertion of quantization nodes. Special quantization operations are inserted in the network to simulate the INT8 behavior of weights and activations.
3. Calibration (PTQ) or training (QAT).
4. Generation of a deployable quantized model

QAT and PTQ in Vitis AI

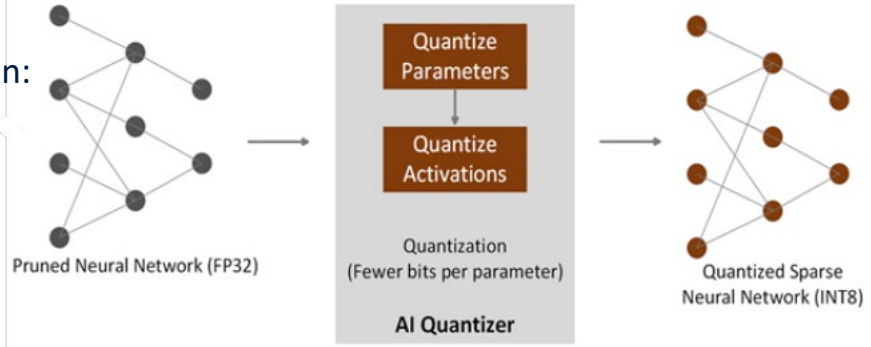
Depending on the method used, the VitisQuantizer class can:

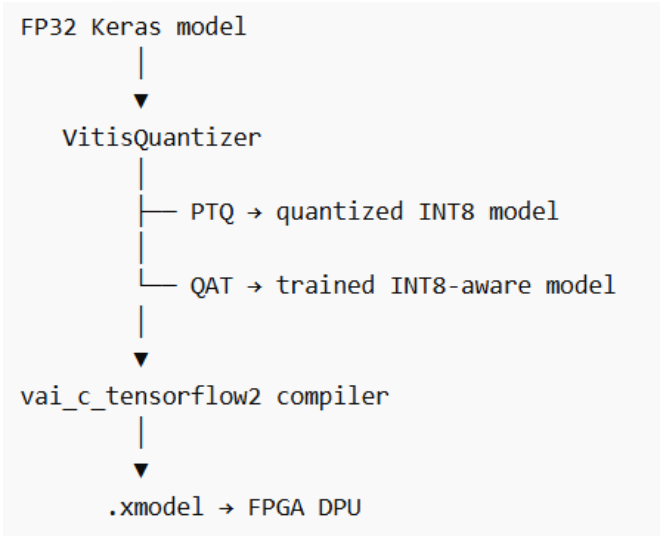
- Perform PTQ, using **.quantize_model()**
- Enable QAT using **.get_qat_model()**

In Vitis AI, several quantization strategies are available:
For istance:

- "**fs**" - Uses floating-point scaling factors (as in our previous examples).
- "**pof2s**" - Uses Power-of-two scaling. This is the default strategy designed for the DPU (and the only one supported).

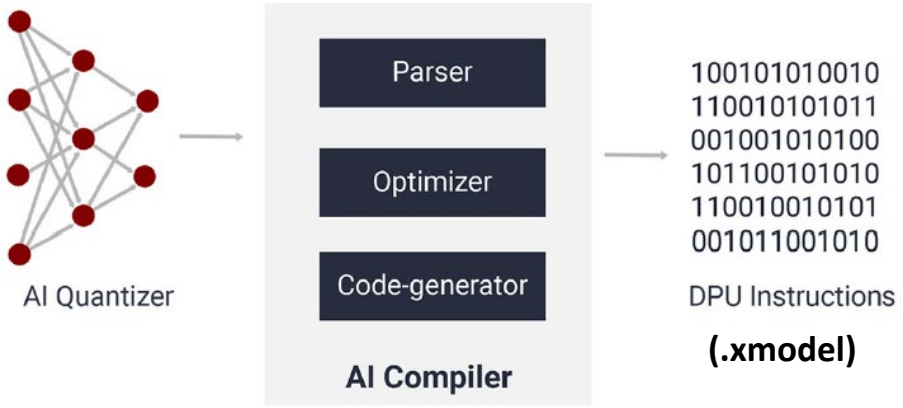
More on this in tomorrow's tutorial!





Vitis AI Compiler

- The Vitis AI Compiler (VAI_C) converts a quantized neural network into instructions executable by the DPU. VAI_C is actually a family of compilers for different DPUs.
- The model is first transformed into XIR (Xilinx Intermediate Representation), a hardware-aware graph representation used internally by the compiler.
- The compiler performs graph parsing, optimization, and scheduling, applying transformations such as operator fusion, memory optimization, and layer mapping to the target DPU architecture.



Using the **target architecture description (arch.json)**, the compiler generates **DPU instruction streams** and produces the final deployable model file: **.xmodel**

Takeaways

- DPU
- Floating point number and IEEE 754 Floating-Point Standard
- The importance of quantization
- Symmetric vs Asymmetric quantization
- PTQ and QAT
- Vitis AI toolchain